

文档编号	TY-SUM-20231030-A
版 本	A
页 数	56 页

系统建模仿真环境 内核算法替换用户手册

科学计算与系统建模仿真项目组

编制	田显钊	生效日期	2023/10/30
审核	郭俊峰	批准	刘奇

文件变更摘要

日期	版本	变更说明	修订	审核	批准
2023/10/30	V1	初始建立	田显钊	郭俊峰	刘奇

苏州同元软控信息技术有限公司版权所有，未经许可不得复制、传播或以其他方式使用；
版权所有，侵权必究。

目 录

1. 模型求解算法	1
1.1 ODE 初值问题求解算法	3
1.1.1 数据结构.....	4
1.1.2 算法函数.....	8
1.1.3 工具函数.....	13
1.1.4 计算函数.....	15
1.1.5 算法注册接口.....	17
1.2 线性代数系统.....	18
1.2.1 数据结构.....	18
1.2.2 算法函数.....	22
1.2.3 工具函数.....	25
1.2.4 算法注册接口.....	27
1.3 非线性代数系统.....	28
1.3.1 数据结构.....	28
1.3.2 算法函数.....	32
1.3.3 计算函数.....	35
1.3.4 工具函数.....	37
1.3.5 算法注册接口.....	39
2. 算法替换案例	39
2.1 算法函数实现.....	40
2.2 算法注册.....	47
2.3 算法使用.....	51
2.4 运行已有案例.....	54

1. 模型求解算法

科学计算与系统建模仿真平台模型翻译后的方程系统包括 ODE 初值问题、线性代数系统、非线性代数系统。模型仿真需要对这些方程系统进行解算，因此模型求解用到 ODE 初值问题求解算法、线性系统求解算法、非线性系统求解算法。科学计算与系统建模仿真平台内置有很多这类求解算法，除此之外，平台还提供扩展模型求解算法的功能。

对科学计算与系统建模仿真平台模型求解在主控层的对外接口包括创建仿真实例、算法注册、模型仿真、销毁实例。其中，算法注册用于集成算法层扩展的求解算法。

模型仿真在主控层内部分为初始化、单步计算和仿真结束三个步骤。单步计算循环执行，使仿真在时间轴上推进，直至满足仿真终止条件而结束仿真。

ODE 初值问题、线性代数系统、非线性代数系统的求解算法函数在单步计算过程中被调用。当模型中有此类方程时，主控层调用算法层的算法函数对方程进行解算。算法层的算法函数在解算过程中，为获得方程的描述而调用主控层提供的计算函数（初值问题和非线性问题）或从主控层获取数据（线性问题）。

后续将详细说明扩展科学计算与系统建模仿真平台模型求解算法涉及到的主控层求解算法注册接口、方程的计算函数原型或数据格式、工具函数的原型，以及算法层算法函数原型。

苏州同元软控信息技术有限公司
版权所有，侵权必究。

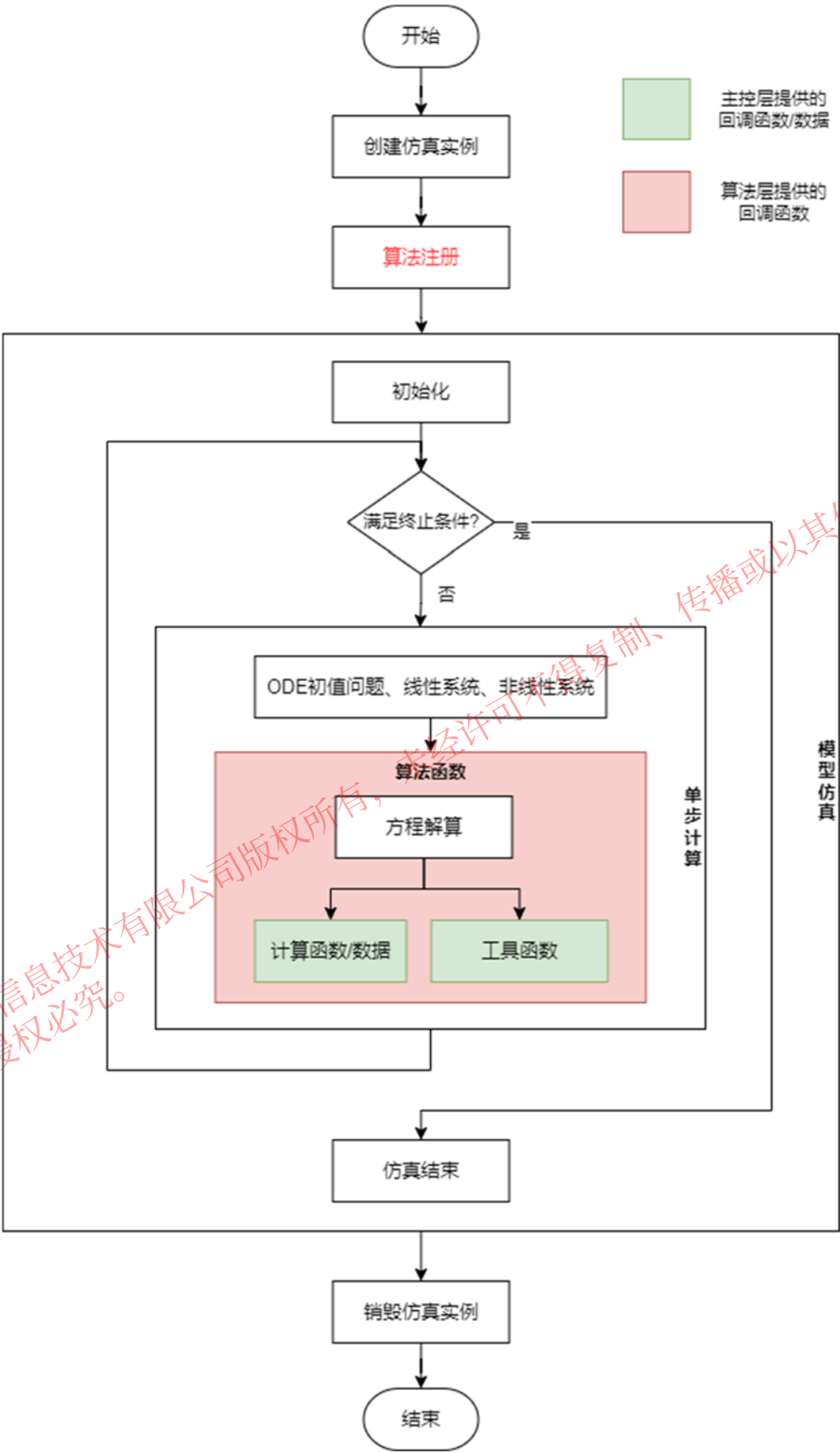


图 1-1 求解器与算法交互过程

模型求解算法扩展的步骤是相同的：

- (1). 首先，按规范实现算法函数
- (2). 然后在应用层将这些算法函数通过算法注册/设置接口集成到求解器。
- (3). 模型方程和算法函数一起构建出模型的求解器，用于对模型进行仿真。

ODE 初值问题求解算法是注册到系统，每次仿真时在界面中选取指定名字的算法，即 ODE 初值问题求解算法注册后具备被选作求解算法的资格。

线性代数系统和非线性代数系统的求解算法是设置到系统，成为每次仿真时使用的算法。



图 1-2 模型求解算法扩展过程

求解源文件 `mws_user_algo.c` 中有 `MwsRegisterUserAlgorithm1`、`MwsRegisterUserAlgorithm2` 函数，在前一函数中注册和设置线性系统求解算法、非线性系统求解算法，在后一函数中注册 ODE 初值问题求解算法。

1.1 ODE 初值问题求解算法

模型的 ODE 初值问题通用形式为：

$$M\dot{y} = f(t, y)$$

$$y(t_0) = y_0$$

其中， t 是自变量， y 是因变量， $\dot{y}$ 是 $\frac{dy}{dt}$ 。 M 是变换矩阵，大多数情况下是单位阵。

另外，一些求解算法还将右端表示为两部分。例如：

$$M\dot{y} = f^E(t, y) + f^I(t, y)$$

$$y(t_0) = y_0$$

其中， $f^E(t, y)$ 表示非刚性（nonstiff）部分，使用显式方法求解； $f^I(t, y)$ 表示

刚性（stiff）部分，使用隐式方法求解。

显式问题：

$$\dot{y} = f(t, y)$$

隐式问题：

$$f(t, y, \dot{y}) = 0$$

初值问题的算法本文也称之为积分算法，正在运行的积分算法对象称之为积分器。

1.1.1 数据结构

数据结构由规范定义，算法扩展时可直接使用。

函数名	简介
typedef void* MwsIVPSolverObj;	积分算法对象
typedef void* MwsIVPObj;	求解问题对象
typedef enum MwsIVPStatus	积分算法函数返回状态
typedef struct MwsIVPOptions	积分算法选项
typedef enum MwsIVPType;	积分算法求解问题类型
typedef struct MwsIVPSolverFcns	积分算法使用的接口函数
typedef struct MwsIVPUtilFcns	积分算法使用的回调函数
typedef struct MwsIVPCallback	积分算法使用的回调函数

1.1.1.1 积分算法对象

typedef void* MwsIVPSolverObj;	
功能	积分算法对象

1.1.1.2 求解问题对象

typedef void* MwsIVPObj;

功能	求解问题对象
----	--------

1.1.1.3 积分算法函数返回状态

```
typedef enum
{
    MWS_IVP_SUCCESS = 0,
    MWS_IVP_WARNING = 1,
    MWS_IVP_TSTOP_RETURN,
    MWS_IVP_MEM_FAIL,
    MWS_IVP_FAIL,
    MWS_IVP_INVALID_INPUT,
    MWS_IVP_RHSFN_FAIL,
    MWS_IVP_RESFN_FAIL,
}MwsIVPStatus;
```

功能	积分算法函数返回状态
枚举项	MWS_IVP_SUCCESS = 0, 成功 MWS_IVP_WARNING = 1, 有警告 MWS_IVP_TSTOP_RETURN, 到达终止时间而返回 MWS_IVP_MEM_FAIL, 内存分配失败 MWS_IVP_FAIL, 失败（原因未知） MWS_IVP_INVALID_INPUT, 输入无效 MWS_IVP_RHSFN_FAIL, ODE 通用形式中右端函数出错 MWS_IVP_RESFN_FAIL, DAE 残余函数出错

1.1.1.4 积分算法选项

```
typedef struct MwsIVPOptions
{
    MwsBoolean m_stopTimeDefined;
    MwsReal m_stopTime;
    MwsBoolean m_toleranceDefined;
    MwsReal* m_relativeTolerance;
    MwsReal* m_absoluteTolerance;
    MwsBoolean m_maxStepSizeDefined;
```



```
MwsReal m_maxStepSize;
}MwsIVPOptions;
```

功能	积分算法选项	
属性	m_stopTimeDefined;	终止时间是否定义
	m_stopTime;	终止时间
	m_toleranceDefined;	容许误差是否定义
	m_relativeTolerance;	相对容许误差，数组
	m_absoluteTolerance;	绝对容许误差，数组
	m_maxStepSizeDefined;	最大步长是否定义
	m_maxStepSize;	最大步长

1.1.1.5 积分算法求解问题类型

```
typedef enum
{
    MWS_IVP_ODE,
    MWS_IVP_DAE,
}MwsIVPType;
```

功能	积分算法求解问题类型	
枚举项	MWS_IVP_ODE,	ODE 问题
	MWS_IVP_DAE,	DAE 问题

1.1.1.6 积分算法使用的算法函数

```
typedef struct MwsIVPSolverFcns
{
    MwsIVPSolverCreatePtr    m_createPtr;
    MwsIVPCreatePtr          m_createPBPtr;
    MwsIVPInitPtr            m_initPtr;
    MwsIVPSolvePtr           m_solvePtr;
    MwsIVPInterpolatePtr     m_interpolatePtr;
    MwsIVPDestroyPtr         m_destroyPBPtr;
    MwsIVPSolverDestroyPtr   m_destroyPtr;
```

```
}MwsIVPSolverFcns;
```

功能	积分算法使用的算法函数	
属性	m_createPtr;	对象创建的函数指针
	m_createPBPtr;	问题对象创建的函数指针
	m_initPtr;	初始化的函数指针
	m_solvePtr;	求解的函数指针
	m_interpolatePtr;	插值的函数指针
	m_destroyPBPtr;	问题对象销毁的函数指针
	m_destroyPtr;	对象销毁的函数指针

1.1.1.7 积分算法使用的工具函数

```
typedef struct
{
    void (*m_logger)(void* user_data, MwsInteger error_code, MwsString where, MwsString msg);

    void* (*m_allocMemory)(void* user_data, MwsSize nobj, MwsSize size);

    void (*m_freeMemory)(void* user_data, void* p);

    void* (*m_allocDataMemory)(void* user_data, MwsSize nobj, MwsSize size);

    void (*m_freeDataMemory)(void* user_data, void* p);
}MwsIVPUtilFcns;
```

功能	积分算法使用的工具函数	
属性	m_logger	日志函数
	m_allocMemory	内存分配函数
	m_freeMemory	内存释放函数
	m_allocDataMemory	数据内存分配函数
	m_freeDataMemory	数据内存释放函数

1.1.1.8 积分算法使用的计算函数

```
typedef struct
{
    MwsIVPRshFcnPtr      m_rshFunction;
    MwsIVPResFcnPtr      m_resFunction;
    MwsIVPJacFcnPtr      m_jacFunction;
    MwsIVPStepFinishedPtr m_stepFinished;
} MwsIVPCallback;
```

功能	积分算法使用的计算函数	
属性	m_rshFunction	ODE 右端函数
	m_resFunction	DAE 残余函数（与 m_rshFunction 互斥）
	m_jacFunction	Jacobian 计算函数
	m_stepFinished	积分步完成回调函数

1.1.2 算法函数

算法函数由算法扩展方实现，即实现 ODE 初值问题求解算法，其中必须实现的函数由主控层按如下流程进行调用。

苏州同元软控信息技术有限公司 版权所有，未经许可不得复制、传播或以其他方式使用；
版权所有，侵权必究。

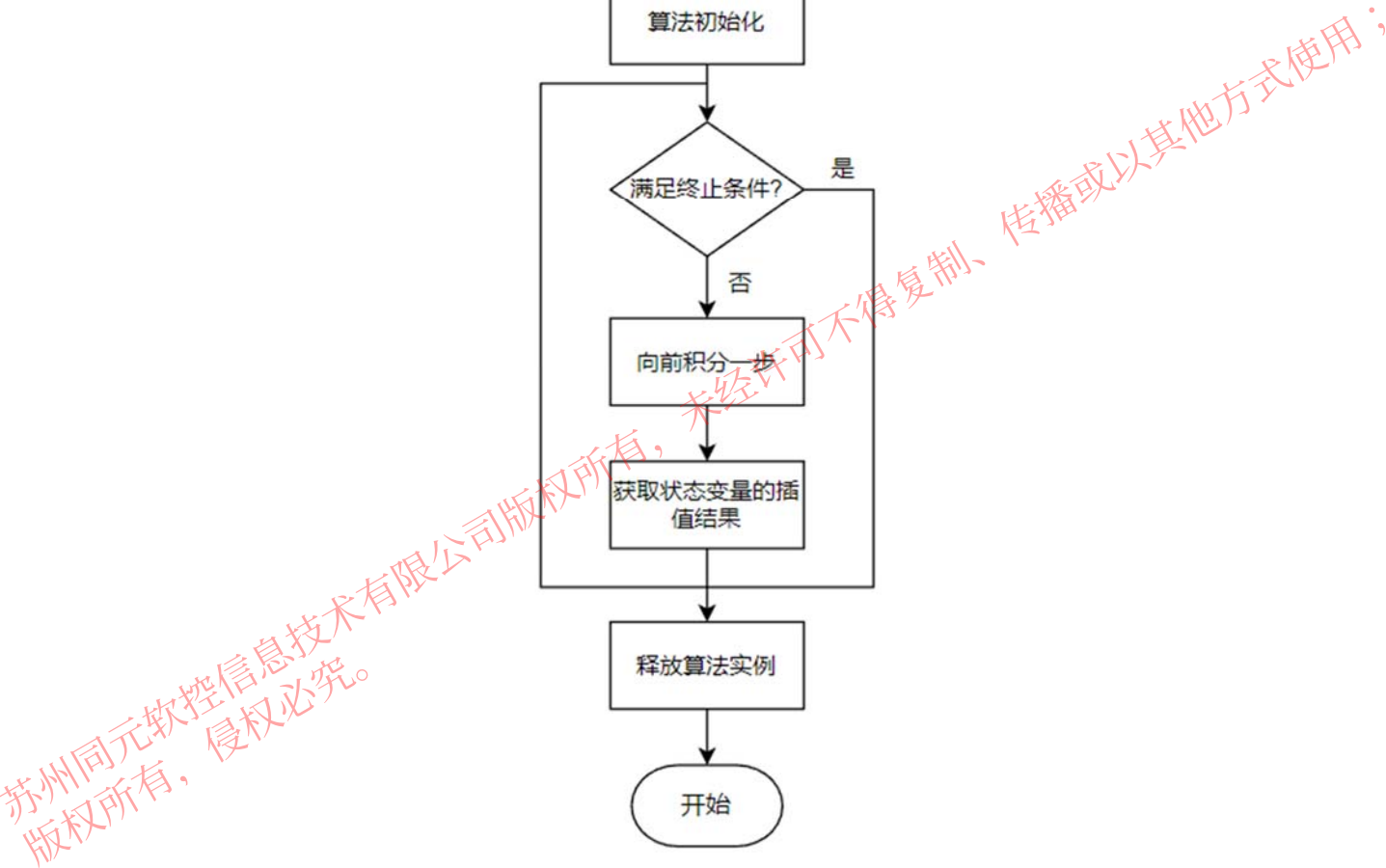


图 1-3 主控层调用算法函数过程

函数名	简介
MwsIVPSolverCreatePtr	积分算法对象创建函数
MwsIVPCreatePtr	求解问题对象创建函数
MwsIVPInitPtr	积分算法初始化
MwsIVPSolvePtr	求解函数

MwsIVPInterpolatePtr	插值函数
MwsIVPDestroyPtr	求解问题对象销毁函数
MwsIVPSolverDestroyPtr	释放积分算法实例

1.1.2.1 积分算法对象创建函数

```
typedef MwsIVPSolverObj (*MwsIVPSolverCreatePtr)(
MwsIVPUtilFcns* util_fcns,
void* user_data);
```

功能	创建积分算法对象
参数	[in]util_fcns 工具函数 [in]user_data 用户数据(传递给工具函数)
返回	积分算法对象
备注	工具函数必定会提供

1.1.2.2 求解问题对象创建函数

```
typedef MwsIVPObj (*MwsIVPCreatePtr)(
MwsIVPSolverObj solver,
MwsSize n,
MwsIVPCallback* call_back,
MwsIVPOptions* opt,
void* ivp_user_data);
```

功能	获取积分算法的特性
参数	[in]solver 积分算法对象 [in]n 规模，状态变量数量 [in]call_back 计算函数 [in]opt 积分算法选项 [in]ivp_user_data 用户数据(传递给回调函数)

返回	求解问题对象
备注	回调函数必定会提供

1.1.2.3 积分算法初始化

```
typedef MwsInteger (*MwsIVPInitPtr)(
MwsIVPSolverObj solver,
MwsIVPObj ivp,
MwsReal t0,
const MwsReal* y0,
const MwsReal* yp0,
MwsBoolean is_reinit,
void* reserve);
```

功能	积分算法初始化
参数	[in]solver 积分算法对象 [in]ivp 求解问题对象 [in]t0 起始时间 [in]y0 y 的初始值（必定不为空） [in]yp0 y'的初始值（DAE） [in]is_reinit 是否重新初始化（重启） [in]reserve 保留参数，暂不使用
返回	状态，取 MwsIVPStatus 的值

1.1.2.4 求解函数

```
typedef MwsInteger (*MwsIVPSolvePtr)(
MwsIVPSolverObj solver,
MwsIVPObj ivp,
MwsReal step_size,
MwsReal t,
MwsReal tout,
MwsReal* tret,
MwsReal* yret,
MwsReal* ypret,
void* reserve);
```

功能	求解函数
参数	[in]solver 积分算法对象 [in]ivp 求解问题对象 [in]step_size 步长（定步长/初始化积分步长） [in]t 当前时间 [in]tout 期望的输出时间 [out]tret 求解实际到达的时间 [out]yret y 的结果值 [out]ypret y'的结果值（DAE） [in]reserve 保留参数，暂不使用 * @return 状态，取 MwsIVPStatus 的值
返回	状态，取 MwsIVPStatus 的值

1.1.2.5 插值函数

```
typedef MwsInteger (*MwsIVPInterpolatePtr)(
MwsIVPSolverObj solver,
MwsIVPObj ivp,
MwsReal tout,
MwsReal* yret,
void* reserve);
```

功能	插值函数
参数	[in]solver 积分算法对象 [in]ivp 求解问题对象 [in]tout 期望的输出时间 [out]yret y 的结果值 [in]reserve 保留参数，暂不使用
返回	状态，取 MwsIVPStatus 的值

1.1.2.6 求解问题对象销毁函数

```
typedef void (*MwsIVPDestroyPtr)(
```

```
MwsIVPSolverObj solver,
MwsIVPObj ivp);
```

功能	求解问题对象销毁函数
参数	[in]solver 积分算法对象 [in]ivp 求解问题对象

1.1.2.7 积分算法对象销毁函数

```
typedef void (*MwsIVPSolverDestroyPtr)(MwsIVPSolverObj solver);
```

功能	释放积分算法实例
参数	[in]solver 积分算法对象

1.1.3 工具函数

工具函数由主控层提供，在算法层由算法函数调用。

函数名	简介
m_logger	日志函数
m_allocMemory	内存分配函数
m_freeMemory	内存释放函数
m_allocDataMemory	数据内存分配函数
m_freeDataMemory	数据内存释放函数

1.1.3.1 日志函数

```
void (*m_logger)(void* user_data, MwsInteger error_code, MwsString where, MwsString msg);
```

功能	日志函数
参数	[in] user_data 用户数据，创建对象接口的最后一个参数，下

	同
[in] error_code	错误号
[in] where	错误来自何处
[in] msg	错误信息

1.1.3.2 内存分配函数

```
void* (*m_allocMemory)(void* user_data, MwsSize nobj, MwsSize size);
```

功能	内存分配函数
参数	[in] user_data 用户数据 [in] nobj 对象数量 [in] size 对象大小（字节数）
返回	成功则返回分配的首地址，否则返回空

1.1.3.3 内存释放函数

```
void* (*m_allocMemory)(void* user_data, MwsSize nobj, MwsSize size);
```

功能	内存释放函数
参数	[in] user_data 用户数据 [in] p 内存首地址

1.1.3.4 数据内存分配函数

```
void* (*m_allocDataMemory)(void* user_data, MwsSize nobj, MwsSize size);
```

功能	数据内存分配函数
参数	[in] user_data 用户数据

	[in] nobj 对象数量 [in] size 对象大小（字节数）
返回	成功则返回分配的首地址，否则返回空

1.1.3.5 数据内存释放函数

```
void (*m_freeDataMemory)(void* user_data, void* p);
```

功能	数据内存释放函数
参数	[in] user_data 用户数据 [in] p 内存首地址

1.1.4 计算函数

计算函数由主控层提供，在算法层由算法接口函数调用。

计算函数由主控层提供，其中 ODE 模型函数值表示下面两种形式中的 f 函数值，在算法层由算法函数调用。

显式问题：

$$\dot{y} = f(t, y)$$

隐式问题：

$$f(t, y, \dot{y}) = 0$$

函数名	简介
MwsIVPRshFcnPtr	ODE 通用形式右端函数 $y'=f(t,y)$
MwsIVPResFcnPtr	DAE 残余量计算回调函数
MwsIVPJacFcnPtr	Jacobian 计算回调函数
MwsIVPStepFinishedPtr	积分步完成回调函数

1.1.4.1 ODE 通用形式右端计算函数 $y'=f(t,y)$

```
typedef MwsInteger (*MwsIVPRshFcnPtr)(void* user_data, MwsReal t, const MwsReal* y,
```

```
MwsReal* yp);
```

功能	ODE 通用形式右端计算函数 $y'=f(t,y)$
参数	[in]user_data 用户数据，创建问题对象接口的最后一个参数，下同 [in]t 时间 [in]y y 的值 [out]yp y'的值
返回	状态，MwsIVPStatus 类型的值

1.1.4.2 DAE 残余量计算函数

```
typedef MwsInteger (*MwsIVPResFcnPtr)(void* user_data, MwsReal t, const MwsReal* y, const MwsReal* yp, MwsReal* delta);
```

功能	DAE 残余量计算函数
参数	[in]user_data 用户数据 [in]t 时间 [in]y y 的值 [in]yp y'的值 [out]delta 残余量
返回	状态，MwsIVPStatus 类型的值

1.1.4.3 Jacobian 计算函数

```
typedef MwsInteger(*MwsIVPJacFcnPtr)(void* user_data, MwsReal t, const MwsReal* y, const MwsReal* yp, MwsReal cj, MwsReal* pd);
```

功能	Jacobian 计算函数
----	---------------

参数	[in]user_data 用户数据 [in]t 时间 [in]y y 的值 [in]yp y'的值 (DAE) [in]cj 作用于 Jacobian 的标量 (DAE) [out]pd 偏导 (pd=dF/dy+cj*dF/dy'或 pd=df/dy)
返回	状态, MwsIVPStatus 类型的值

1.1.4.4 积分步完成函数

```
typedef MwsInteger (*MwsIVPStepFinishedPtr)(void* user_data, MwsReal t, const MwsReal* y);
```

功能	积分步完成回调函数
参数	[in]user_data 用户数据 [in]t 时间 [in]y y 的值
返回	状态, MwsIVPStatus 类型的值

1.1.5 算法注册接口

注册接口通常在 mws_user_algo.c 文件中调用，对实现的算法进行注册。

函数名	简介
isimRegisterIVPSolver	注册积分算法
isimUnregisterIVPSolver	移除注册的积分算法

1.1.5.1 注册积分算法

```
MoBoolean isimRegisterIVPSolver(void* sim_data, const MwsIVPSolverProp* prop, const MwsIVPSolverFcns* fcns);
```

功能	注册积分算法
参数	[in] sim_data 单步层数据 [in] prop 积分算法属性 [in] fcns 积分算法函数

1.1.5.2 移除注册的积分算法

```
MoBoolean isimUnregisterIVPSolver(void* sim_data, MwsString name);
```

功能	移除注册的积分算法
参数	[in] sim_data 单步层数据 [in] name 积分算法名字

1.2 线性代数系统

线性代数系统的形式为：

$Ax = b$

其中 A 是系数矩阵，b 是常数项，x 是未知量。

1.2.1 数据结构

数据结构由规范定义，算法扩展时可直接使用。

函数名	简介
typedef void* MwsLSSolverObj;	线性代数系统求解算法对象
typedef void* MwsLSPObj;	线性代数系统求解问题对象
typedef enum MwsLSStatus	线性代数系统求解算法函数返回状态
typedef enum MwsLSFactorType	线性代数系统系数类型
typedef struct MwsLSSolverProp	线性代数系统求解算法属性
typedef struct MwsLSOptions	线性代数系统求解选项

typedef struct MwsLSSolverFcns	线性代数系统求解算法函数
typedef struct MwsLSUtilFcns	线性代数系统求解算法工具函数

1.2.1.1 线性求解算法对象

```
typedef void* MwsLSSolverObj;
```

功能	线性代数系统求解算法对象
----	--------------

1.2.1.2 线性求解问题对象

```
typedef void* MwsLSPObj;
```

功能	线性代数系统求解问题对象
----	--------------

1.2.1.3 线性代数系统求解算法函数返回状态

```
typedef enum
{
    MWS_LS_SUCCESS,
    MWS_LS_WARNING,
    MWS_LS_MEM_FAIL,
    MWS_LS_FAIL,
    MWS_LS_INVALID_INPUT,
    MWS_LS_SINGULAR_WARNING,
    MWS_LS_SINGULAR_ERROR,
}MwsLSStatus;
```

功能	线性代数系统求解算法函数返回状态	
枚举项	MWS_LS_SUCCESS,	成功
	MWS_LS_WARNING,	有警告
	MWS_LS_MEM_FAIL,	内存分配失败
	MWS_LS_FAIL,	失败（原因未知）

	MWS_LS_INVALID_INPUT,	输入无效
	MWS_LS_SINGULAR_WARNING,	线性系统相容奇异
	MWS_LS_SINGULAR_ERROR,	线性系统矛盾奇异

1.2.1.4 线性代数系统系数类型

```
typedef enum
{
    LS_FACTOR_DEFAULT,
    LS_FACTOR_STATIC_A,
    LS_FACTOR_A_UPDATE,
    LS_FACTOR_b_UPDATE,
}MwsLSFactorType;
```

功能	线性代数系统系数类型
枚举项	LS_FACTOR_DEFAULT, 缺省，A、b 更新 LS_FACTOR_STATIC_A, A 固定不变 LS_FACTOR_A_UPDATE, 只更新 A LS_FACTOR_b_UPDATE, 只更新 b

1.2.1.5 线性代数系统求解算法属性

```
typedef struct MwsLSSolverProp
{
    MwsString m_name;
    MwsString m_desc;
}MwsLSSolverProp;
```

功能	线性代数系统求解算法属性
属性	m_name; 名称（不区分大小写），保证唯一 m_desc; 算法描述

1.2.1.6 线性代数系统求解选项

```
typedef struct
{
    MwsBoolean m_toleranceDefined; 容许误差是否定义
    MwsReal m_tolerance;           相对容许误差，数组
}MwsLSOptions;
```

功能	线性代数系统求解选项
属性	m_toleranceDefined; 容许误差是否定义 m_tolerance; 相对容许误差，数组

1.2.1.7 线性代数系统求解算法函数

```
typedef struct MwsLSSolverFcns
{
    MwsLSSolverCreatePtr m_createPtr;
    MwsLSProblemCreatePtr m_createPBPtr;
    MwsLSSolvePtr m_solvePtr;
    MwsLSProblemDestroyPtr m_destroyPBPtr;
    MwsLSSolverDestroyPtr m_destroyPtr;
}MwsLSSolverFcns;
```

功能	线性代数系统求解算法函数
属性	m_createPtr; 对象创建的函数指针 m_createPBPtr; 问题对象创建的函数指针 m_solvePtr; 求解的函数指针 m_destroyPBPtr; 问题对象销毁的函数指针 m_destroyPtr; 对象销毁的函数指针

1.2.1.8 线性代数系统求解算法工具函数

```
typedef struct
{
```



```
void (*m_logger)(void* user_data, MwsInteger error_code, MwsString where, MwsString
msg);

void* (*m_allocMemory)(void* user_data, MwsSize nobj, MwsSize size);

void (*m_freeMemory)(void* user_data, void* p);

void* (*m_allocDataMemory)(void* user_data, MwsSize nobj, MwsSize size);

void (*m_freeDataMemory)(void* user_data, void* p);
}MwsLSUtilFcns;
```

功能	线性代数系统求解算法使用的工具函数	
属性	m_logger	日志函数
	m_allocMemory	内存分配函数
	m_freeMemory	内存释放函数
	m_allocDataMemory	数据内存分配函数
	m_freeDataMemory	数据内存释放函数

1.2.2 算法函数

线性代数系统求解算法的算法函数由扩展方实现，其中必须实现的函数由主控层调用的主体流程如下：

苏州同元软控信息技术有限公司版权所有，未经许可，不得复制、传播或以其他方式使用；
版权所有，侵权必究。



图 1-4 主控层调用算法函数过程

函数名	简介
MwsLSSolverCreatePtr	线性代数系统求解算法对象创建函数
MwsLSProblemCreatePtr	线性代数系统求解算法对象创建函数
MwsLSSolvePtr	求解函数
MwsLSProblemDestroyPtr	求解问题对象销毁函数
MwsLSSolverDestroyPtr	求解对象销毁函数

1.2.2.1 线性代数系统求解算法对象创建函数

```
typedef MwsLSSolverObj(*MwsLSSolverCreatePtr)(MwsLSUtilFcns* util_fcns, void* user_data);
```

功能	线性代数系统求解算法对象创建函数
参数	[in]util_fcns 工具函数

	[in]user_data 用户数据
返回	线性求解算法对象

1.2.2.2 线性代数系统求解问题对象创建函数

```
typedef MwsLSPObj (*MwsLSProblemCreatePtr)(
MwsLSSolverObj solver,
MwsSize n,
MwsLSFactorType factor_type,
MwsBoolean tol_defined,
MwsReal tol,
const MwsReal* a,
const MwsReal* b,
void* lsp_user_data);
```

功能	线性代数系统求解问题对象创建函数
参数	[in]solver 线性求解算法对象 [in]n A 的阶数 [in]factor_type 系数特征 [in]tol_defined 是否定义了误差 [in]tol 误差 [in]a A [in]b b [in]lsp_user_data 求解函数等可使用

1.2.2.3 求解函数

```
typedef MwsInteger(*MwsLSSolvePtr)(MwsLSSolverObj solver, MwsLSPObj lsp, const MwsReal*
a,
const MwsReal* b, MwsReal* xret, void* reserve);
```

功能	求解函数
参数	[in]solver 线性代数系统求解算法对象 [in]lsp 求解问题对象

	[in]aA
	[in]bb
	[out]xretx 的结果值
	[in]reserve保留参数，暂不使用
返回	状态，取 MwsLSStatus 的值

1.2.2.4 求解问题对象销毁函数

```
typedef void (*MwsLSProblemDestroyPtr)(MwsLSSolverObj solver, MwsLSPObj lsp);
```

功能	求解问题对象销毁函数
参数	[in]solver线性代数系统求解算法对象 [in]lsp求解问题对象

1.2.2.5 求解对象销毁函数

```
typedef void (*MwsLSSolverDestroyPtr)(MwsLSSolverObj solver);
```

功能	求解对象销毁函数
参数	[in]solver线性求解算法对象

1.2.3 工具函数

工具函数由主控层提供，在算法层由算法函数调用。

函数名	简介
m_logger	日志函数
m_allocMemory	内存分配函数
m_freeMemory	内存释放函数
m_allocDataMemory	数据内存分配函数
m_freeDataMemory	数据内存释放函数

1.2.3.1 日志函数

```
void (*m_logger)(void* user_data, MwsInteger error_code, MwsString where, MwsString msg);
```

功能	日志函数	
参数	[in] user_data	用户数据，创建对象接口的最后一个参数，下同
	[in] error_code	错误号
	[in] where	错误来自何处
	[in] msg	错误信息

1.2.3.2 内存分配函数

```
void* (*m_allocMemory)(void* user_data, MwsSize nobj, MwsSize size);
```

功能	内存分配函数	
参数	[in] user_data	用户数据
	[in] nobj	对象数量
	[in] size	对象大小（字节数）
返回	成功则返回分配的首地址，否则返回空	

1.2.3.3 内存释放函数

```
void* (*m_allocMemory)(void* user_data, MwsSize nobj, MwsSize size);
```

功能	内存释放函数	
参数	[in] user_data	用户数据
	[in] p	内存首地址

1.2.3.4 数据内存分配函数

```
void* (*m_allocDataMemory)(void* user_data, MwsSize nobj, MwsSize size);
```

功能	数据内存分配函数
参数	[in] user_data 用户数据 [in] nobj 对象数量 [in] size 对象大小（字节数）
返回	成功则返回分配的首地址，否则返回空

1.2.3.5 数据内存释放函数

```
void (*m_freeDataMemory)(void* user_data, void* p);
```

功能	数据内存释放函数
参数	[in] user_data 用户数据 [in] p 内存首地址

1.2.4 算法注册接口

注册接口通常在 mws_user_algo.c 文件中调用，对实现的算法进行注册。

函数名	简介
imdlRegisterLSSolver	注册线性代数系统求解算法
imdlUnregisterLSSolver	移除注册的线性代数系统求解算法

1.2.4.1 注册线性代数系统求解算法

```
MoBoolean imdlRegisterLSSolver(void* mdl_data, const MwsLSSolverProp* prop, const MwsLSSolverFcns* fcns);
```

功能	注册线性代数系统求解算法
参数	[in]mdl_data 模型层数据 [in]prop 线性代数系统求解算法属性 [in]fcns 线性代数系统求解算法函数

1.2.4.2 移除注册的线性代数系统求解算法

```
MoBoolean imdlUnregisterLSSolver(void* mdl_data, MwsString name);
```

功能	移除注册的线性代数系统求解算法
参数	[in]mdl_data 模型层数据 [in]name 线性代数系统求解算法名字

1.3 非线性代数系统

非线性代数系统的形式为：

$$F(x) = 0$$

其中 x 是未知量。

1.3.1 数据结构

数据结构由规范定义，算法扩展时可直接使用。

函数名	简介
typedef void* MwsNLSSolverObj;	非线性代数系统求解算法对象
typedef void* MwsNLSPObj;	非线性代数系统求解问题对象
typedef enum MwsNLSStatus	非线性代数系统求解算法函数返回状态
typedef struct MwsNLSSolverProp	非线性代数系统求解算法属性
typedef struct MwsNLSSOptions	非线性代数系统求解选项

typedef struct MwsNLSSolverFcns	非线性代数系统求解算法接口函数
typedef struct MwsNLSUtilFcns	非线性代数系统求解算法工具函数

1.3.1.1 非线性代数系统求解算法对象

```
typedef void* MwsNLSSolverObj;
```

功能	非线性代数系统求解算法对象
----	---------------

1.3.1.2 非线性代数系统求解问题对象

```
typedef void* MwsNLSPObj;
```

功能	非线性代数系统求解问题对象
----	---------------

1.3.1.3 非线性代数系统求解算法函数返回状态

```
typedef enum
{
    MWS_NLS_SUCCESS,
    MWS_NLS_WARNING,
    MWS_NLS_MEM_FAIL,
    MWS_NLS_FAIL,
    MWS_NLS_INVALID_INPUT,
    MWS_NLS_MAX_F_EVAL,
    MWS_NLS_TOO_SMALL_TOL,
    MWS_NLS_JAC_NO_IMPROVE,
    MWS_NLS_RES_NO_IMPROVE,
    MWS_NLS_ABNORMAL_NUM,
    MWS_NLS_RES_FCN_FAIL,
    MWS_NLS_JAC_FCN_FAIL,
}MwsNLSStatus;
```

功能	非线性代数系统求解算法函数返回状态
----	-------------------

枚举项	MWS_NLS_SUCCESS	成功
	MWS_NLS_WARNING	有警告
	MWS_NLS_MEM_FAIL	内存分配失败
	MWS_NLS_FAIL	失败（原因未知）
	MWS_NLS_INVALID_INPUT	输入无效
	MWS_NLS_MAX_F_EVAL	F 调用到达限定的次数
	MWS_NLS_TOO_SMALL_TOL	设置太小，迭代无法提高精度
	MWS_NLS_JAC_NO_IMPROVE	连续 5 次更新 Jacobian 矩阵， 结果没有改进
	MWS_NLS_RES_NO_IMPROVE	连续 10 次迭代，结果没有改进
	MWS_NLS_ABNORMAL_NUM	计算过程中出现了异常数
	MWS_NLS_RES_FCN_FAIL	残余函数失败
	MWS_NLS_JAC_FCN_FAUL	雅可比计算函数失败

1.3.1.4 非线性代数系统求解算法属性

```
typedef struct MwsNLSSolverProp
{
    MwsString m_name;
    MwsString m_desc;
}MwsNLSSolverProp;
```

功能	非线性代数系统求解算法属性
属性	m_name; 名称（不区分大小写），保证唯一 m_desc; 算法描述

1.3.1.5 线性代数系统求解选项

```
typedef struct
{
    MwsBoolean m_toleranceDefined;    容许误差是否定义
    MwsReal m_tolerance;              相对容许误差，数组
}MwsNLSOptions;
```

功能	线性代数系统求解选项
属性	m_toleranceDefined; 容许误差是否定义 m_tolerance; 相对容许误差，数组

1.3.1.6 非线性代数系统求解算法函数

```
typedef struct MwsLSSolverFcns
{
    MwsNLSSolverCreatePtr m_createPtr;
    MwsNLSPbProblemCreatePtr m_createPBPtr;
    MwsNLSSolvePtr m_solvePtr;
    MwsNLSPbProblemDestroyPtr m_destroyPBPtr;
    MwsNLSSolverDestroyPtr m_destroyPtr;
}MwsLSSolverFcns;
```

功能	线性代数系统求解算法函数
属性	m_createPtr; 对象创建的函数指针 m_createPBPtr; 问题对象创建的函数指针 m_solvePtr; 求解的函数指针 m_destroyPBPtr; 问题对象销毁的函数指针 m_destroyPtr; 对象销毁的函数指针

1.3.1.7 非线性代数系统求解算法工具函数

```
typedef struct
{
    void (*m_logger)(void* user_data, MwsInteger error_code, MwsString where, MwsString msg);

    void* (*m_allocMemory)(void* user_data, MwsSize nobj, MwsSize size);

    void (*m_freeMemory)(void* user_data, void* p);

    void* (*m_allocDataMemory)(void* user_data, MwsSize nobj, MwsSize size);
```

```
void (*m_freeDataMemory)(void* user_data, void* p);
}MwsNLSUtilFcns;
```

功能	非线性代数系统算法使用的工具函数	
属性	m_logger	日志函数
	m_allocMemory	内存分配函数
	m_freeMemory	内存释放函数
	m_allocDataMemory	数据内存分配函数
	m_freeDataMemory	数据内存释放函数

1.3.2 算法函数

非线性代数系统求解算法函数由扩展方实现，其中必须实现的函数由主控层调用的主体流程如下：

苏州同元软控信息技术有限公司版权所有，未经许可不得复制、传播或以其他方式使用；
版权所有，侵权必究。



图 1-5 主控层调用算法函数过程

函数名	简介
MwsNLSSolverCreatePtr	非线性代数系统求解算法对象创建函数
MwsNLSPProblemCreatePtr	非线性代数系统求解问题对象创建函数
MwsNLSSolvePtr	求解函数
MwsNLSPProblemDestroyPtr	求解问题对象销毁函数
MwsNLSSolverDestroyPtr	求解对象销毁函数

1.3.2.1 非线性代数系统求解算法对象创建函数

```
typedef MwsNLSSolverObj(*MwsNLSSolverCreatePtr)(MwsNLSUtilFcns* util_fcns, void* user_data);
```

功能	非线性代数系统求解算法对象创建函数
参数	[in]util_fcns 工具函数 [in]user_data 用户数据
返回	非线性求解算法对象

1.3.2.2 非线性代数系统求解问题对象创建函数

```
typedef MwsNLSPObj (*MwsNLSPProblemCreatePtr)(MwsNLSSolverObj solver, MwsSize n, MwsNLSSysFnPtr sys_fcn, MwsNLSSysFnPtr jac_fcn, MwsBoolean tol_defined, MwsReal tol, void* nlsp_user_data);
```

功能	非线性代数系统求解问题对象创建函数
参数	[in]solver 求解器对象 [in]n 规模 [in]sys_fcn 非线性系统残余回调函数 [in]jac_fcn Jacobian 计算回调函数 [in]tol_defined 容许误差是否定义 [in]tol 相对容许误差，数组 [in]nlsp_user_data 用户数据(传递给回调函数)
返回	非线性求解问题对象

1.3.2.3 求解函数

```
typedef MwsInteger(*MwsNLSSolvePtr)(MwsNLSSolverObj solver, MwsNLSPObj nlsp, MwsString const * var_names, const MwsReal* u, MwsReal* u_nominal,
```

```
MwsReal* uret,  
void* reserve);
```

功能	求解函数	
参数	[in]solver	非线性代数系统求解算法对象
	[in]nlsp	求解问题对象
	[in]var_names	变量名数组（用于信息输出）
	[in]u	u 的初始估算值
	[in out]u_nominal	u 的 nominal 值
	[out]uret	u 的结果值
	[in]reserve	保留参数，暂不使用
返回	状态，取 MwsNLSStatus 的值	

1.3.2.4 求解问题对象销毁函数

```
typedef void (*MwsNLSPProblemDestroyPtr)(MwsNLSSolverObj solver, MwsNLSPObj nlsp);
```

功能	求解问题对象销毁函数	
参数	[in]solver	非线性代数系统求解算法对象
	[in]lsp	求解问题对象

1.3.2.5 求解对象销毁函数

```
typedef void (*MwsNLSSolverDestroyPtr)(MwsNLSSolverObj solver);
```

功能	求解对象销毁函数	
参数	[in]solver	非线性代数系统求解算法对象

1.3.3 计算函数

计算函数由主控层提供，其中，非线性代数系统函数值计算函数表示非线性方程

$F(x) = 0$

中的函数 F。

函数名	简介
MwsNLSSysFnPtr	计算 F(u)的值， $F(u) = 0$
MwsNLPJacEvaluation	计算 F(u)的 Jacobian

1.3.3.1 计算 F(u)的值， $F(u) = 0$

```
typedef MwsInteger (*MwsNLSSysFnPtr)(void* nlsp_user_data, const MwsReal* u, MwsReal* fret, void* reserve);
```

功能	计算 F(u)的值， $F(u) = 0$
参数	[in]nlsp_user_data 用户数据 [in]u u 的值 [out]fret F 的值 [in]reserve 保留参数，暂不使用
返回	非线性代数系统求解状态

1.3.3.2 计算 F(u)的 Jacobian

```
typedef MwsInteger (*MwsNLPJacEvaluation)(MwsUserContext ctx, MwsReal x[], MwsReal jac[]);
```

功能	计算 F(u)的 Jacobian
参数	[in]nlsp_user_data 用户数据 [in]u u 的值 [out]jac Jacobian [in]reserve 保留参数，暂不使用
返回	非线性代数系统求解状态

1.3.4 工具函数

工具函数由主控层提供，算法层根据需要调用。

函数名	简介
m_logger	日志函数
m_allocMemory	内存分配函数
m_freeMemory	内存释放函数
m_allocDataMemory	数据内存分配函数
m_freeDataMemory	数据内存释放函数

1.3.4.1 日志函数

```
void (*m_logger)(void* user_data, MwsInteger error_code, MwsString where, MwsString msg);
```

功能	日志函数
参数	[in] user_data 用户数据，创建对象接口的最后一个参数，下同 [in] error_code 错误号 [in] where 错误来自何处 [in] msg 错误信息

1.3.4.2 内存分配函数

```
void* (*m_allocMemory)(void* user_data, MwsSize nobj, MwsSize size);
```

功能	内存分配函数
参数	[in] user_data 用户数据 [in] nobj 对象数量 [in] size 对象大小（字节数）

返回	成功则返回分配的首地址，否则返回空
----	-------------------

1.3.4.3 内存释放函数

```
void* (*m_allocMemory)(void* user_data, MwsSize nobj, MwsSize size);
```

功能	内存释放函数
参数	[in] user_data 用户数据 [in] p 内存首地址

1.3.4.4 数据内存分配函数

```
void* (*m_allocDataMemory)(void* user_data, MwsSize nobj, MwsSize size);
```

功能	数据内存分配函数
参数	[in] user_data 用户数据 [in] nobj 对象数量 [in] size 对象大小（字节数）
返回	成功则返回分配的首地址，否则返回空

1.3.4.5 数据内存释放函数

```
void (*m_freeDataMemory)(void* user_data, void* p);
```

功能	数据内存释放函数
参数	[in] user_data 用户数据 [in] p 内存首地址

1.3.5 算法注册接口

注册接口通常在 mws_user_algo.c 文件中调用，对实现的算法进行注册。

函数名	简介
imdlRegisterNLSSolver	注册非线性代数系统求解算法
imdlUnregisterNLSSolver	移除注册的非线性代数系统求解算法

1.3.5.1 注册非线性代数系统求解算法

```
MoBoolean imdlRegisterNLSSolver(void* mdl_data, const MwsNLSSolverProp* prop, const MwsNLSSolverFcns* fcns);
```

功能	注册非线性代数系统求解算法
参数	[in]mdl_data 模型层数据 [in]prop 线性代数系统求解算法属性 [in]fcns 线性代数系统求解算法函数

1.3.5.2 移除注册的非线性代数系统求解算法

```
MoBoolean imdlUnregisterNLSSolver(void* mdl_data, MwsString name);
```

功能	移除注册的非线性代数系统求解算法
参数	[in]mdl_data 模型层数据 [in]name 非线性代数系统求解算法名字

2. 算法替换案例

科学计算与系统建模仿真平台模型翻译后的方程系统包括 ODE 初值问题、

线性代数系统、非线性代数系统。模型仿真需要对这些方程系统进行解算，因此模型求解用到 ODE 初值问题求解算法、线性系统求解算法、非线性系统求解算法。科学计算与系统建模仿真平台内置有很多这类求解算法，除此之外，平台支持用户自定义并扩展求解算法。

用户自定义扩展求解算法主要流程参见 1 中的说明及图 1-2 的流程图。

根据模型求解器算法扩展流程，以下案例详细说明增加自定义显示欧拉积分算法并使用该算法进行求解；

2.1 算法函数实现

由于积分算法与其它求解算法的计算过程不同，因此，针对不同的算法，用户需要提供不同的算法函数，此案例主要演示积分算法的主要算法函数及使用。

A. 创建 c 文件并引入头文件

该示例中，创建一个 my_euler.c 文件，并引入规范定义的算法函数头文件：

```
#include "mo_types.h"
#include "mws_ivp_solver.h"
```

如图所示：

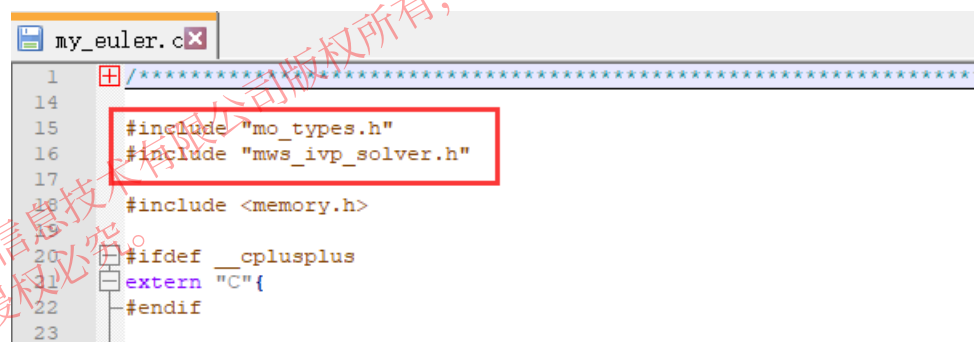


图 2-1 创建文件并增加头文件

以下接口均在该 c 文件中进行编写。

B. 数据结构定义

数据结构用来保存求解器传入的工具函数、计算函数、自定义用户数据等，以及算法内部使用的数据。算法的数据结构在规范中没有任何规定或限制，完全由算法开发者自主实现。

```
/* 算法对象 */
typedef struct
{
```

```

    MwsIVUtilFcns    m_utils;

    void*            m_userData;

} MyEuler;

/* 求解问题的数据，用数据内存分配和释放函数 */
typedef struct
{
    MoReal *m_preY;
    MoReal *m_curY;

    MoReal m_curTime;
    MoReal m_initialStep;
} MyEulerProblemData;

/* 求解问题对象 */
typedef struct
{
    MoSize      m_nStates;

    MwsIVOptions m_opt;
    MwsIVPCallback m_callback;
    void*       m_userData;

    MyEulerProblemData* m_data;
    MyEuler* m_solverWork;
} MyEulerProblem;

```

C. 编写积分算法对象创建函数

(1). 函数形式:

```
typedef MwsIVPSolverObj (*MwsIVPSolverCreatePtr)(MwsIVUtilFcns* util_fcns, void* user_data);
```

(2). 输入:

- **util_fcns**: 工具函数（由求解器提供，算法函数中可调用）
- **user_data**: 用户数据（求解器内部数据，传递给工具函数 **util_fcns**，自定义算法无需关心其内容）

(3). 返回:

积分算法对象(自定义算法内部数据，求解器不关心其内容)，作为其它算法函数的第一个参数。

(4). my_euler.c 文件中的示例

```

MwsIVPSolverObj myEulerCreate(MwsIVPUtilFcns* util_fcns, void* user_data)
{
    MyEuler* sw = (MyEuler*)util_fcns->m_allocMemory(user_data, 1, sizeof(MyEuler));
    if (sw)
    {
        memset(sw, 0, sizeof(*sw));
        sw->m_utils = *util_fcns;
        sw->m_userData = user_data;
    }
    return sw;
}

```

D. 编写求解问题对象创建函数

(1). 函数形式:

```

typedef MwsIVPObj (*MwsIVPCreatePtr)( MwsIVPSolverObj solver, MwsSize n, MwsIVPCallback* call_back,
MwsIVPOptions* opt, void* ivp_user_data);

```

(2). 输入:

- **solver**: 积分算法对象
- **n**: 问题规模, 即状态变量数量
- **call_back**: 计算函数
- **opt**: 积分器选项
- **ivp_user_data**: 用户数据(求解器内部数据, 传递给回调函数 **call_back**, 算法不需关心)

(3). 返回:

问题对象(自定义算法内部数据, 求解器不关心其内容), 作为其它算函数的第二个参数。

(4). my_euler.c 文件中的示例:

```

MwsIVPObj myEulerProblemCreate(MwsIVPSolverObj solver, MwsSize n, MwsIVPCallback* call_back,
MwsIVPOptions* opt, void* ivp_user_data)
{
    MyEuler* sw = (MyEuler*)solver;
    MyEulerProblem* spw = (MyEulerProblem*)sw->m_utils.m_allocMemory(sw->m_userData, 1,
sizeof(MyEulerProblem));
    if (spw)
    {
        MyEulerProblemData* ds =
(MyEulerProblemData*)sw->m_utils.m_allocDataMemory(sw->m_userData, 1, sizeof(MyEulerProblemData));
        if (ds == mwsNullPtr)

```

```

    {
        sw->m_utils.m_freeMemory(sw->m_userData, spw);
        return mwsNullPtr;
    }

    memset(spw, 0, sizeof(*spw));
    memset(ds, 0, sizeof(*ds));
    spw->m_callback = *call_back;
    spw->m_userData = ivp_user_data;
    spw->m_nStates = n;
    spw->m_data = ds;
    spw->m_solverWork = sw;
    spw->m_data->m_curTime = 0;
    spw->m_data->m_initialStep = 0.002;
    if (spw->m_nStates > 0)
    {
        spw->m_data->m_preY = (MoReal *)sw->m_utils.m_allocDataMemory(sw->m_userData, 1,
n*sizeof(MoReal));
        spw->m_data->m_curY = (MoReal *)sw->m_utils.m_allocDataMemory(sw->m_userData, 1,
n*sizeof(MoReal));
        if (!spw->m_data->m_preY || !spw->m_data->m_curY)
        {
            myEulerProblemDestroy(sw, spw);
            spw = MWnullptr;
        }
        else
        {
            MoSize index;
            for (index=0; index<n; ++index)
            {
                spw->m_data->m_preY[index] = 0;
                spw->m_data->m_curY[index] = 0;
            }
        }
    }
    return spw;
}

```

E. 编写积分算法初始化函数

(1). 函数形式:

```

typedef MwsInteger (*MwsIVPInitPtr)(MwsIVPSolverObj solver, MwsIVPObj ivp, MwsReal t0, const MwsReal*
y0, const MwsReal* yp0, MwsBoolean is_reinit, void* reserve);

```

(2). 输入:

- **solver**: 积分算法对象
- **ivp**: 问题对象
- **t0**: 起始时间
- **y0**: y 的初始值 (必定不为空)
- **yp0**: y'的初始值 (DAE)
- **is_reinit**: 是否重新初始化 (重启)
- **reserve**: 保留参数, 暂不使用

(3). 返回: 状态, 取 **MwsIVPStatus** 的值。

(4). **my_euler.c** 文件中的示例:

```
MwsInteger myEulerInit(MwsIVPSolverObj solver, MwsIVPObj ivp, MwsReal t0, const MwsReal* y0,
    const MwsReal* yp0, MwsBoolean is_reinit, void* reserve)
{
    /* nothing to do */
    return MWS_IVP_SUCCESS;
}
```

F. 编写求解函数

(1). 函数形式:

```
typedef MwsInteger (*MwsIVPSolvePtr)(MwsIVPSolverObj solver, MwsIVPObj ivp, MwsReal step_size,
MwsReal t, MwsReal tout, MwsReal* tret, MwsReal* yret, MwsReal* ypret, void* reserve);
```

(2). 输入:

- **solver**: 积分算法对象;
- **ivp**: 问题对象;
- **step_size**: 步长 (定步长/初始化积分步长);
- **t**: 当前时间;
- **tout**: 期望的输出时间;
- **reserve**: 保留参数, 暂不使用。

(3). 输出:

- **tret**: 求解实际达到的时间;
- **yret**: y 的结果值;
- **ypret**: y'的结果值 (DAE)。

(4). 返回: 状态, 取 **MwsIVPStatus** 的值。

(5). **my_euler.c** 文件中的示例:

```
MwsInteger myEulerSolve(MwsIVPSolverObj solver, MwsIVPObj ivp, MwsReal step_size, MwsReal t, MwsReal
tout, MwsReal* tret, MwsReal* yret, MwsReal* ypret, void* reserve)
```

```

{
    MyEuler* sw = (MyEuler*)solver;
    MyEulerProblem* spw = (MyEulerProblem*)ivp;

    MoSize index = 0;
    MoSize nState = spw->m_nStates;

    MoReal* preY = spw->m_data->m_preY;
    MoReal* curY = spw->m_data->m_curY;

    spw->m_data->m_curTime = t;
    spw->m_data->m_initialStep = step_size;    for (index = 0; index<nState; ++index)
    {
        preY[index] = yret[index];
        yret[index] += spw->m_data->m_initialStep * ypret[index];
        curY[index] = yret[index];
    }

    /* 更新当前积分时间 */
    *tret = tout;

    return MWS_IVP_SUCCESS;
}

```

G. 编写插值函数

(1). 函数形式:

```

typedef MwsInteger (*MwsIVPInterpolatePtr)(MwsIVPSolverObj solver, MwsIVPObj ivp, MwsReal tout,
MwsReal* yret, void* reserve);

```

(2). 输入:

- **solver**: 积分算法对象;
- **ivp**: 问题对象;
- **tout**: 期望的输出时间;
- **reserve**: 保留参数, 暂不使用。

(3). 输出:

- **yret**: y 的结果值。

(4). 返回: 状态, 取 **MwsIVPStatus** 的值。

(5). my_euler.c 文件中的示例:

```

MwsInteger myEulerInterpolate(MwsIVPSolverObj solver, MwsIVPObj ivp, MwsReal tout, MwsReal* yret,
void* reserve)

```



```

{
    /* 线性插值 */
    MyEuler* sw = (MyEuler*)solver;
    MyEulerProblem* spw = (MyEulerProblem*)ivp;
    MoSize nStates = spw->m_nStates;
    MoSize index;
    for (index=0; index<nStates; ++index)
    {
        /* 线性插值:
            
$$L(x) = \frac{f(x_2)-f(x_1)}{x_2-x_1} * (x - x_1) + f(x_1)$$

        */
        yret[index] = (spw->m_data->m_curY[index] - spw->m_data->m_preY[index]) * (
            tout - spw->m_data->m_curTime + spw->m_data->m_initialStep) / spw->m_data->m_initialStep +
            spw->m_data->m_preY[index];
    }
    return MWS_IVP_SUCCESS;
}

```

H. 编写求解问题对象销毁函数

(1). 函数形式:

```
typedef void (*MwsIVPDestroyPtr)(MwsIVPSolverObj solver, MwsIVPObj ivp);
```

(2). 输入:

- **solver**: 积分算法对象
- **ivp**: 问题对象

(3). my_euler.c 文件中的示例:

```

void myEulerProblemDestroy(MwsIVPSolverObj solver, MwsIVPObj ivp)
{
    MyEuler* sw = (MyEuler*)solver;
    MyEulerProblem* spw = (MyEulerProblem*)ivp;
    if (spw)
    {
        if (spw->m_data->m_preY)
        {
            (*sw->m_utils.m_freeDataMemory)(sw->m_userData, spw->m_data->m_preY);
        }
        if (spw->m_data->m_curY)
        {
            (*sw->m_utils.m_freeDataMemory)(sw->m_userData, spw->m_data->m_curY);
        }
    }
}

```

```

        if (spw->m_data)
        {
            (*sw->m_utils.m_freeDataMemory)(sw->m_userData, spw->m_data);
        }
        (*sw->m_utils.m_freeMemory)(sw->m_userData, spw);
    }
}

```

I. 编写积分算法对象销毁函数

(1). 函数形式:

```
typedef void (*MwsIVPSolverDestroyPtr)(MwsIVPSolverObj solver);
```

(2). 输入:

- **solver**: 积分算法对象

(3). my_euler.c 文件中的示例:

```

void myEulerDestroy(MwsIVPSolverObj solver)
{
    MyEuler* sw = (MyEuler*)solver;
    if (sw)
    {
        (*sw->m_utils.m_freeMemory)(sw->m_userData, sw);
    }
}

```

2.2 算法注册

要在软件中使用上述自定义算法，需要将算法注册到求解器。

注册文件为【MWorks 安装目录】/simulator/src/mws_user_algo.c。下面以 myeuler 积分算法为例，介绍算法注册方法。

A. 新建 mws_user_alog.c 文件

新建 mws_user_alog.c 文件，因为后面要对 MWorks 目录下的该文件进行替换，因此文件名不能改变。

引入以下头文件：包括科学计算平台提供的标准头文件和自定义的 my_euler.c 头文件。

```

#include "mws_ivp_solver.h"
#include "mws_ls_solver.h"
#include "mws_nls_solver.h"
#include "my_euler.c"

```

如图所示：

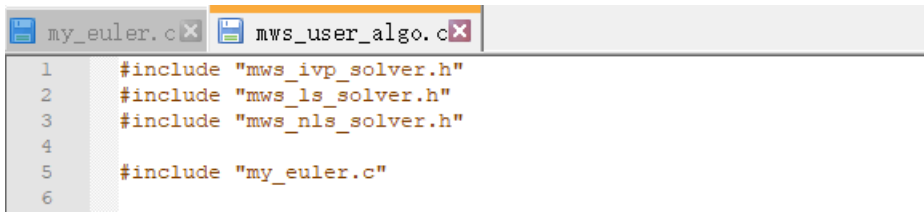


图 2-2 创建 mws_user_algo.c 文件

B. 注册函数介绍

算法注册文件中，共有 4 个函数，其中包括：

- 线性/非线性求解算法注册函数

```
void MwsRegisterUserAlgorithm1(void* mdl_data);
```

- 积分算法注册函数

```
void MwsRegisterUserAlgorithm2(void* sim_data);
```

- 线性/非线性求解算法注销函数

```
void MwsUnregisterUserAlgorithm1(void* mdl_data);
```

- 积分算法注销函数

```
void MwsUnregisterUserAlgorithm2(void* sim_data);
```

C. 注册模板参考

我们提供了一个注册模板，用户可在其基础上进行修改，注册模板如下：

```

#include "mws_ivp_solver.h"
#include "mws_ls_solver.h"
#include "mws_nls_solver.h"
#include "xxxxxx" /* 自定义算法头文件 */
void MwsRegisterUserAlgorithm1(void* mdl_data)
{
    /* Register user defined LS and NLS algorithm. */
    //线性
    MwsLSSolverProp ls_prop;
    MwsLSSolverFcns ls_fcns;
    ls_prop.m_name = "myls"; /* 名称（不区分大小写），保证唯一 */
    ls_prop.m_desc = "myls"; /* 算法描述 */
    ls_fcns.m_createPtr = &myLsCreate; /* 创建算法对象函数指针 */
    ls_fcns.m_createPBPtr = &myLsCreatePB; /* 创建问题对象函数指针 */
    ls_fcns.m_solvePtr = &myLsSolve; /* 求解函数指针 */
    ls_fcns.m_destroyPBPtr = &myLsDestroyPB; /* 销毁问题对象函数指针 */
    ls_fcns.m_destroyPtr = &myLsDestroy; /* 销毁算法对象函数指针 */
    imdlRegisterLSSolver(mdl_data, &ls_prop, &ls_fcns);
}

```

```

//非线性
MwsNLSSolverProp nls_prop;
MwsNLSSolverFcns nls_fcns;
nls_prop.m_name = "mynls";          /* 名称（不区分大小写），保证唯一 */
nls_prop.m_desc = "mynls";          /* 算法描述 */
nls_fcns.m_createPtr = &myNlsCreate; /* 创建算法对象函数指针 */
nls_fcns.m_createPBPtr = &myNlsCreatePB; /* 创建问题对象函数指针 */
nls_fcns.m_solvePtr = &myNlsSolve;    /* 求解函数指针 */
nls_fcns.m_destroyPBPtr = &myNlsDestroyPB; /* 销毁问题对象函数指针 */
nls_fcns.m_destroyPtr = &myNlsDestroy; /* 销毁算法对象函数指针 */
imdlRegisterNLSSolver(mdl_data, nls_prop, nls_fcns);
}

void MwsRegisterUserAlgorithm2(void* sim_data)
{
    /* Register user defined IVP algorithm. */
    //积分
    MwsIVPSolverProp ivp_prop;
    MwsIVPSolverFcns ivp_fcns;
    ivp_prop.m_name = "myeuler";      /* 名称（不区分大小写），保证唯一 */
    ivp_prop.m_desc = "MYEULER";      /* 算法描述 */
    ivp_prop.m_fixedStep = moTrue;     /* 是否为定步长，moTrue 或 moFalse */
    ivp_prop.m_ivpType = MWS_IVP_ODE; /* 积分算法类型，ODE 为 MWS_IVP_ODE，DAE 为 MWS_IVP_DAE */
    ivp_fcns.m_createPtr = &myEulerCreate; /* 创建算法对象函数指针 */
    ivp_fcns.m_createPBPtr = &myEulerProblemCreate; /* 创建问题对象函数指针 */
    ivp_fcns.m_destroyPBPtr = &myEulerProblemDestroy; /* 销毁问题对象函数指针 */
    ivp_fcns.m_destroyPtr = &myEulerDestroy; /* 销毁算法对象函数指针 */
    ivp_fcns.m_initPtr = &myEulerInit; /* 初始化函数指针 */
    ivp_fcns.m_interpolatePtr = &myEulerInterpolate; /* 插值函数指针 */
    ivp_fcns.m_solvePtr = &myEulerSolve; /* 求解函数指针 */
    isimRegisterIVPSolver(sim_data, &ivp_prop, &ivp_fcns);
}

void MwsUnregisterUserAlgorithm1(void* mdl_data)
{
    /* Unregister user defined LS and NLS algorithm. */
    imdlUnregisterLSSolver(mdl_data, "myls");
    imdlUnregisterNLSSolver(mdl_data, "mynls");
}

void MwsUnregisterUserAlgorithm2(void* sim_data)
{

```

```

/* Unregister user defined IVP algorithm. */
isimUnregisterIVPSolver(sim_data, "myeuler");
}

```

其中，包含语句`#include "xxxxxx"`、注册函数中赋值号“=”右边的部分以及注销函数中的算法名是允许定制的

为了保证自定义算法注册成功，需要满足以下条件：

- (1). 除了包含语句`#include "xxxxxx"`与注册函数中赋值号“=”右边的部分，其他部分不允许修改；
- (2). 算法名字不能重复，不能与内置算法相同；
- (3). 注销函数中的算法名需要与注册函数中的相同。

D. 自定义算法注册函数

以 `myeuler` 算法为例，赋值语句 `ivp_fcns.m_createPtr = &myEulerCreate` 表示注册创建算法对象函数，`myEulerCreate` 为创建算法对象函数名（见算法编写章节），以此类推，则 `myeuler` 算法的注册文件最终内容如下：

```

#include "mws_ivp_solver.h"
#include "mws_ls_solver.h"
#include "mws_nls_solver.h"
#include "my_euler.c"
void MwsRegisterUserAlgorithm1(void* mdl_data)
{
    /* Register user defined LS and NLS algorithm. */
}
void MwsRegisterUserAlgorithm2(void* sim_data)
{
    /* Register user defined IVP algorithm. */
    MwsIVPSolverProp ivp_prop;
    MwsIVPSolverFcns ivp_fcns;
    ivp_prop.m_name = "myeuler";
    ivp_prop.m_desc = "MYEULER";
    ivp_prop.m_fixedStep = moTrue;
    ivp_prop.m_ivpType = MWS_IVP_ODE;
    ivp_fcns.m_createPtr = &myEulerCreate;
    ivp_fcns.m_createPBPtr = &myEulerProblemCreate;
    ivp_fcns.m_destroyPBPtr = &myEulerProblemDestroy;
    ivp_fcns.m_destroyPtr = &myEulerDestroy;
    ivp_fcns.m_initPtr = &myEulerInit;
    ivp_fcns.m_interpolatePtr = &myEulerInterpolate;
    ivp_fcns.m_solvePtr = &myEulerSolve;
}

```

```
    isimRegisterIVPSolver(sim_data, &ivp_prop, &ivp_fcms);
}
void MwsUnregisterUserAlgorithm1(void* mdl_data)
{
    /* Unregister user defined LS and NLS algorithm. */
}
void MwsUnregisterUserAlgorithm2(void* sim_data)
{
    /* Unregister user defined IVP algorithm. */
    isimUnregisterIVPSolver(sim_data, "myeuler");
}
```

E. 替换注册文件

将修改好的注册文件移动到安装目录下的 simulator/src 下，并覆盖 mws_user_algo.c 文件，如图所示：

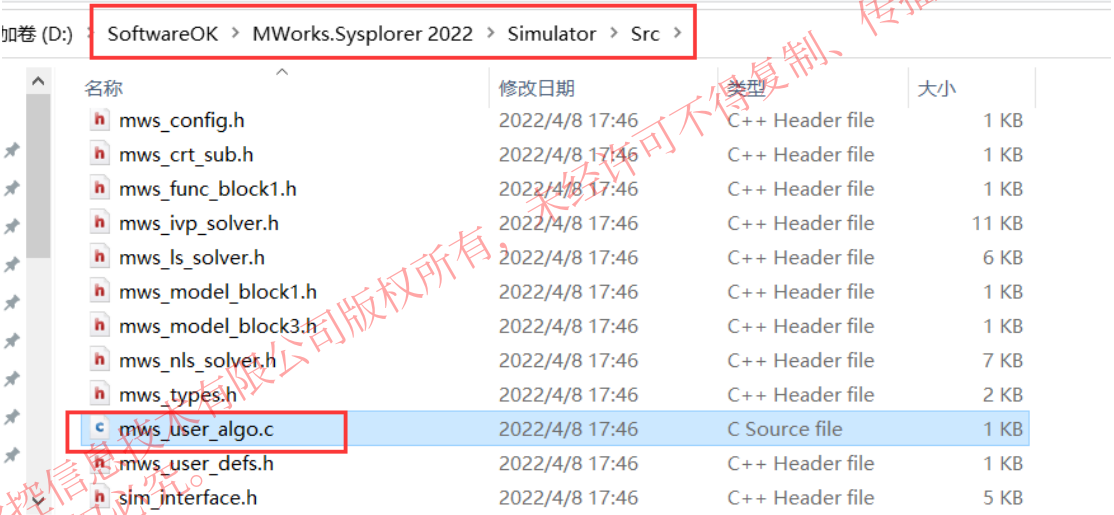


图 2-3 替换 mws_user_algo.c 文件

2.3 算法使用

为了算法编写、注册完成后可在软件中使用，使用自定义算法时需要在软件中做两个工作：链接算法与设置自定义算法。

A. 新建模型

在 MWorks 上新建 ExternInteAlgo 模型，如图所示：

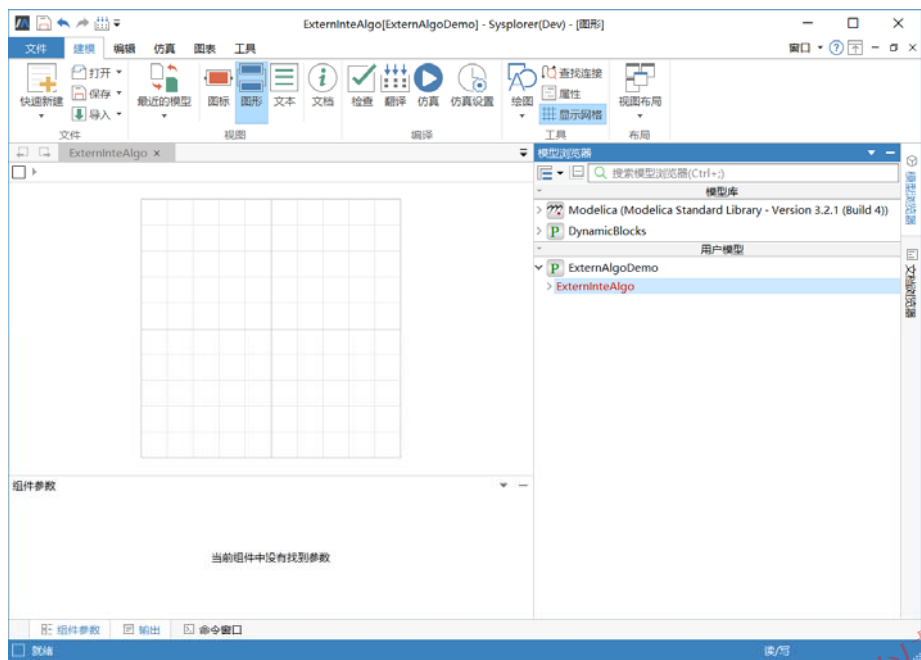


图 2-4 新建模型

并将编写的算法函数接口文件 my_euler.c 文件放在与该模型同一路径下：

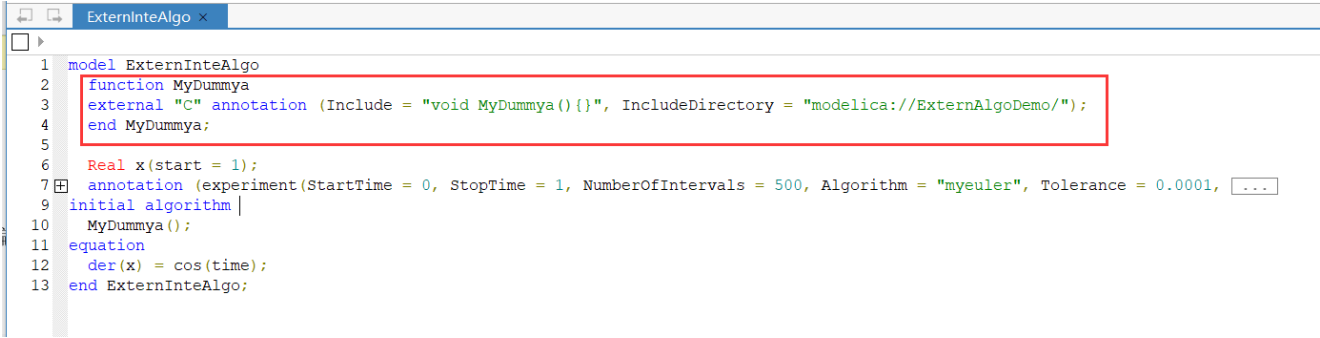
ExternAlgoDemo		
名称		修改日期
ExternAlgo.mo		2023/2/
my_euler.c		2021/2/
package.mo		2021/2/

图 2-5 放置算法函数文件

B. 链接算法

(1). 链接算法的第一步是编写外部函数，其作用是将算法头文件目录写进包含目录。

外部函数的写法如下：



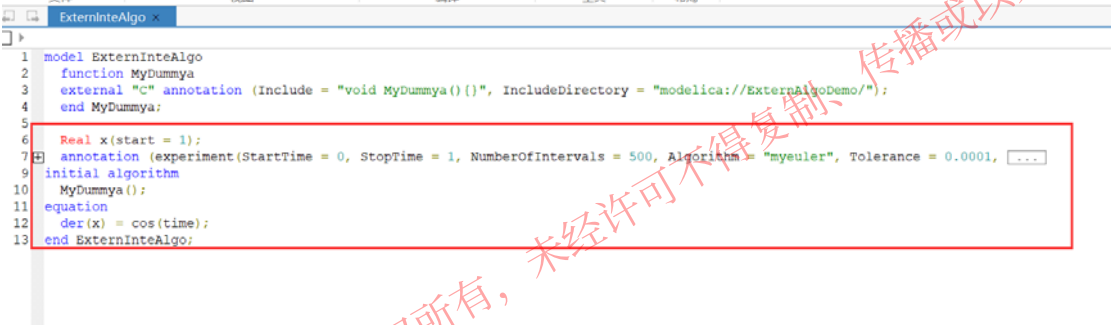
```
1 model ExternInteAlgo
2   function MyDummyya
3     external "C" annotation (Include = "void MyDummyya(){}", IncludeDirectory = "modelica://ExternAlgoDemo/");
4   end MyDummyya;
5
6   Real x(start = 1);
7   annotation (experiment(StartTime = 0, StopTime = 1, NumberOfIntervals = 500, Algorithm = "myeuler", Tolerance = 0.0001, ...));
8
9   initial algorithm
10    MyDummyya();
11  equation
12    der(x) = cos(time);
13  end ExternInteAlgo;
```

图 2-6 编写外部函数

其中，function 声明的函数名要与注解中 Include = "void MyDummyya(){}" 函数名相同，IncludeDirectory 字段的值为头文件路径。

(2). 链接算法的第二步是在模型中调用外部函数，需要写在 initial algorithm 区域。

外部函数调用，如下所示：



```
1 model ExternInteAlgo
2   function MyDummyya
3     external "C" annotation (Include = "void MyDummyya(){}", IncludeDirectory = "modelica://ExternAlgoDemo/");
4   end MyDummyya;
5
6   Real x(start = 1);
7   annotation (experiment(StartTime = 0, StopTime = 1, NumberOfIntervals = 500, Algorithm = "myeuler", Tolerance = 0.0001, ...));
8
9   initial algorithm
10    MyDummyya();
11  equation
12    der(x) = cos(time);
13  end ExternInteAlgo;
```

图 2-7 外部函数调用

C. 设置自定义算法

仿真前，切换到仿真标签页，点击“仿真设置”，在算法下拉框中选择 custom，或直接在菜单栏算法下拉框中选择 custom。

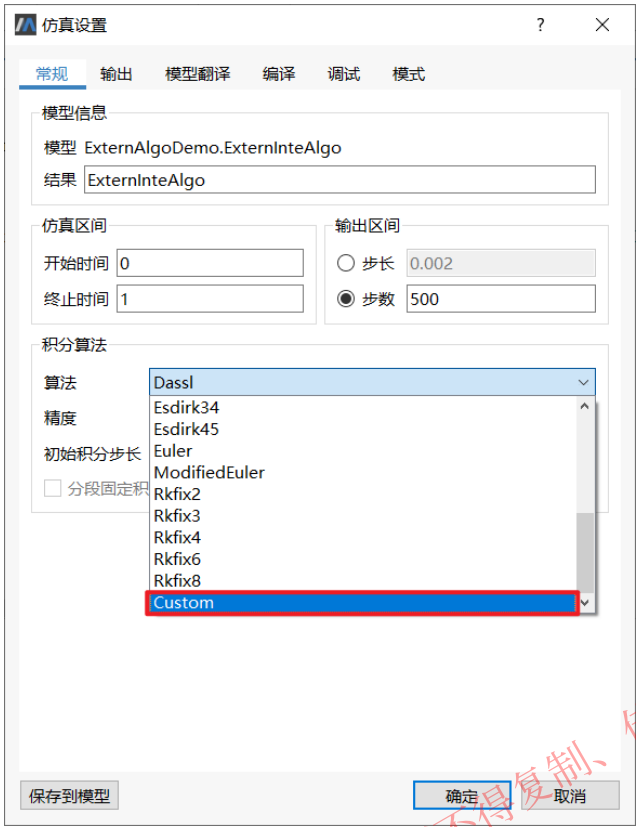


图 2-8 设置自定义算法

点击仿真按钮。此时，若算法的编写、注册、链接都已完成，则将会使用自定义算法进行求解，求解结果如下：

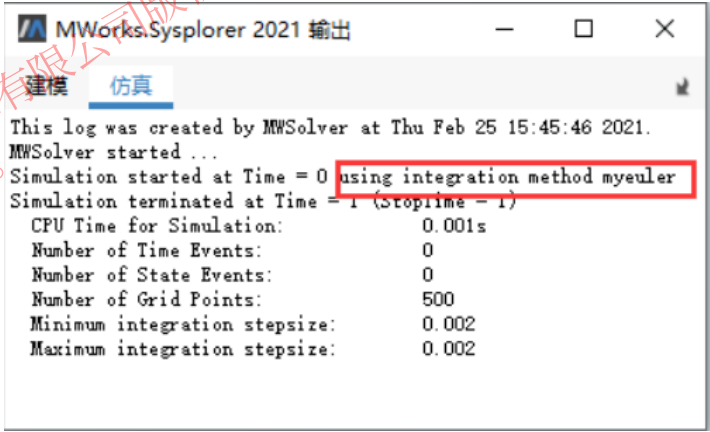


图 2-9 查看求解算法

我们可以看到，已经使用了自定义的 myeuler 算法。

2.4 运行已有案例

打开示例目录 ExternAlgoDemo(【 MWorks 安装路径 】/Docs/static/Samples/ExternAlgoDemo)，其中包含 5 个文件：

 ExternInteAlgo.mo	2021/2/23 9:39	Modelica File	1 KB
 mws_user_algo.c	2021/2/5 15:19	C Source file	2 KB
 my_euler.c	2020/7/7 15:41	C Source file	6 KB
 package.mo	2020/7/7 15:24	Modelica File	1 KB
 说明.txt	2021/2/6 9:55	文本文档	1 KB

图 2-10 示例文件

其中，

- ExternInteAlgo.mo 为模型
- mws_user_algo.c 为算法注册文件
- my_euler.c 为积分算法接口文件
- package.mo 为模型包
- 说明.txt 为使用说明

按照以下步骤使用自定义算法：

A. 拷贝注册文件

将 mws_user_algo.c 拷贝到安装目录下的 Simulator/Src 中覆盖相同文件。

翻译模型生成求解器时，会自动编译这个文件，将注册好的自定义算法生成到求解器中。算法的注册教程请参照算法注册部分。

B. 链接算法

my_euler.c 拷贝至 Simulator/Src，或使用外部函数指定目录。

本例使用外部函数来实现该步骤，打开模型 ExternInteAlgo.mo，外部函数编写在了 ExternInteAlgo.mo 模型中

my_euler.c 文件为自定义算法的接口文件，包含了供求解器调用的自定义算法接口，如创建算法对象、求解等。接口文件的编写教程请参考算法编写部分。

C. 设置自定义算法

仿真前，切换到仿真标签页，点击仿真设置，在算法下拉框中选择 custom，或直接在菜单栏算法下拉框中选择 custom。

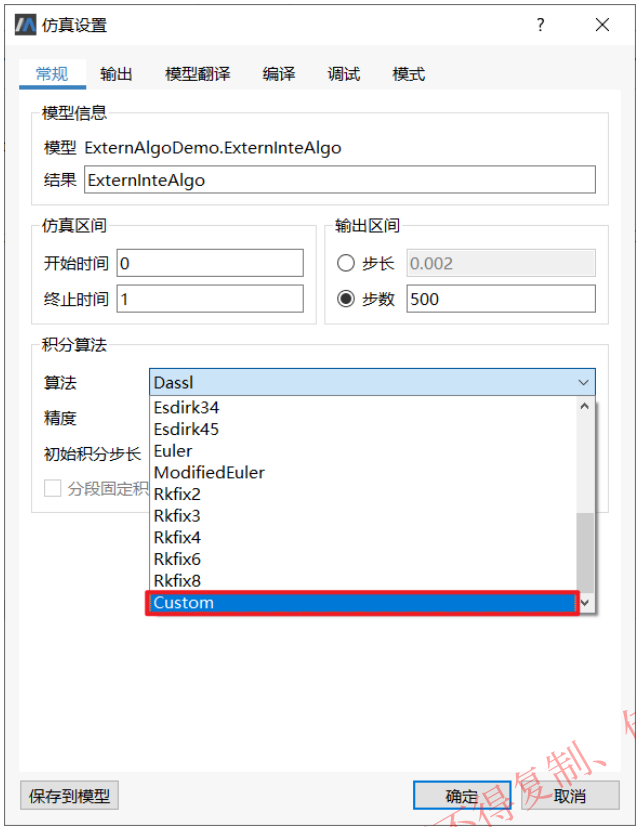


图 2-11 设置自定义算法

点击仿真按钮。此时，若算法的编写、注册、链接都已完成，则将会使用自定义算法进行求解，求解结果如下：

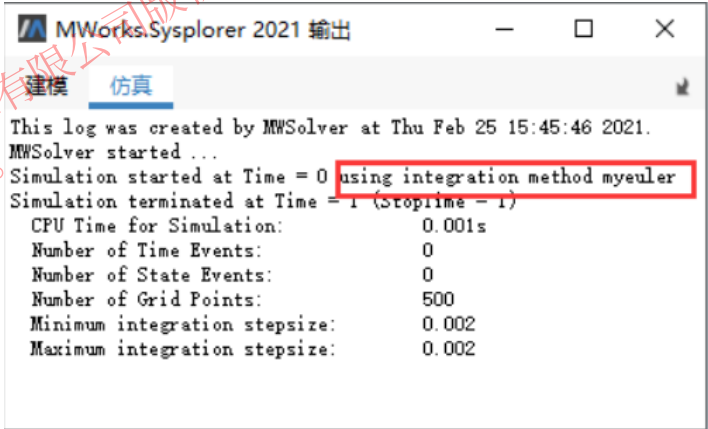


图 2-12 查看求解算法

我们可以看到，已经使用了自定义的 myeuler 算法。